

DandelionDB

Query Language Design Specification

Embedded Vector Database for Edge Devices

Design Rationale: Query Language vs. Function-Based API

Initially, a function-based API seemed like the simpler choice for users, given that the core operations - insert, search, and retrieve - are conceptually simple and primarily operate on vectors. A query language appeared to introduce unnecessary complexity for what is, at its core, a small set of operations.

However, a function-based approach introduces a different problem at the implementation level. Each language binding would need its own function signatures, and any change to the core engine's capabilities would require reimplementing or adapting those functions across every supported language. This makes maintaining consistent APIs across languages significantly harder as the surface area grows.

Introducing a query language with a shared translator - implemented as a DLL (Dynamic Link Library) - resolves this. The core engine exposes a single entry point: a function that accepts a query string and passes it to the DLL for execution. This DLL acts as an intermediary between the engine and every language binding.

With this separation, each language's API only needs to translate its native calls into the query language rather than implement engine logic directly. This translation layer is comparatively simple to build and maintain, and it decouples the engine's capabilities from the complexity of supporting multiple language ecosystems.

This query language will be called the Dandelion Query Language (DQL). With a query language in place, building a functional API for any language becomes straightforward and independent of the core engine's implementation language - each binding only has to generate query text, not reimplement engine behavior.

DQL Syntax Reference

Notation	Meaning
<>	Mandatory name / value / field
[]	Optional field

" "	Non-DB string literal
//	Code comment

Data Definition

Collections

The specification calls a table a collection. This terminology is retained from the current version.

Create Collection

```
create collection "<collection_name>" {
  "<field>" <data_type> [optional_properties],
  "<field>" <data_type> [optional_properties],
  "<field>" <data_type> [optional_properties],
};
```

Example:

```
create collection "company_docs" {
  "doc_id" uint64 primary auto_increment,
  "doc_string" text,
  "doc_embedding" vector<fp32, 768> metric cosine,
  "date_created" datetime,
};

create collection "user_chats" {
  "user_id" uint64 primary auto_increment,
  "chat_room" uint32,
  "message" text,
  "image_sent" bytes nullable,
  "chat_embedding" vector<fp32, 768> metric cosine,
  "chat_deleted" bool,
  "date_created" datetime,
};
```

Drop Collection

```
drop collection "<collection_name>";
```

Dropping a collection removes its collection definition. Data that was associated with the collection becomes unclaimed (or orphaned) storage and may be reclaimed during a later optimization / storage-reclamation process.

Index Management

A collection may contain multiple vector columns. Each vector column may have at most one active vector index. A single CREATE INDEX statement may specify multiple vector columns; this creates an independent index for each specified vector column rather than one combined multi-column vector index.

Creating an Index

```
create index on "<collection>" ("<field>", "<field>", ...)
using <index_type> [with (<index_property> = <value>, ...)];
```

Examples:

```
create index on "company_docs" ("doc_embedding") using ivf;
create index on "company_docs" ("doc_embedding")
using ivf with (nlist = 100);
```

Dropping an Index

```
drop index on "<collection>" ("<field>");
```

Example:

```
drop index on "company_docs" ("doc_embedding");
```

Index Types and Build Properties

Index	Properties	Default
Flat	None - one-by-one exact scan	Exact
HNSW	m, ef_construction	m = 16, ef_construction = 200
IVF	nlist	Automatic

Search Properties for ANN Indexes

Index	Search parameter	Default
Flat	None	-
Index	Search parameter	Default
HNSW	ef	50
IVF	nprobe	1

Compatibility rule: a search property must be valid for the index type used by the target vector column. If an unsupported search property is supplied, the query fails rather than silently ignoring the property.

Data Manipulation

Insert

```
insert into "<collection>" {
  "<field>": <value>,
  "<field>": <value>,
  "<field>": <value>,
  ...
};

insert into "company_docs" {
  "doc_string": "Lorem Ipsum",
  "doc_embedding": [0.1234, 0.2223, 0.3123, ...],
  "date_created": "2026-09-16T10:00:00Z"
};
```

Update

```
update "<collection>"
  set "<field>" = <value>
  filter on <filter_expr>;

update "company_docs"
  set "doc_string" = "Updated Lorem Ipsum"
  filter on "doc_id" = 12;

update "user_chats"
  set "chat_deleted" = true
  filter on "user_id" = 45 and "chat_room" = 3;
```

Delete

```
delete from "<collection>" filter on <filter_expr>;

delete from "company_docs"
  filter on "date_created" < "2026-09-16T00:00:00Z";

delete from "user_chats"
  filter on "chat_deleted" = true;
```

Data Query

Find

```
find <top_clause>
  near <vector_expr>
  on "<collection>" ("<column>")
  [filter on <filter_expr>]
  [search <search_property> <int>]
  return <field_list>;
```

Example:

```
find top 5
  near [0.1234, 0.2223, 0.3123, ...]
  on "company_docs" ("doc_embedding")
  filter on "date_created" > "2026-01-01T00:00:00Z"
  search ef 100
  return "doc_id", "doc_string", _score;

find top 10 within 0.3
  near $query_vector
  on "user_chats" ("chat_embedding")
  filter on "chat_deleted" = false
  return "message", _score;
```

top_clause

Form	Meaning
top <int>	Return at most the specified number of results.
within <float>	Use the native metric of the vector column. - For distance metrics, results must have distance $\leq$ the radius. - For similarity metrics, results must have similarity $\geq$ the threshold.
top <int> within <float>	Return at most <int> results whose metric satisfies the specified radius / threshold.

Current grammar decision: top remains explicit rather than introducing an implicit default-result form. This keeps the grammar small and avoids a hidden query behavior.

vector_expr

Form	Meaning
[<float>, <float>, ...]	=Direct vector value
\$<name>	Placeholder for a value passed as a variable / parameter

Search Property

The optional search clause controls the search-time parameter of the target index. The supported parameter depends on the index type: HNSW uses ef, IVF uses nprobe, and Flat does not accept a search parameter.

Filter Expressions

```
"<field>" <op> <value>
<filter_expr> and <filter_expr>
<filter_expr> or <filter_expr>
```

Examples:

```
"chat_room" > 3
"chat_room" > 3 and "chat_deleted" = false
"chat_room" > 3 or "chat_deleted" = false
```

Operators

Operator	Meaning
=	Exact match
!=	Not equal

<	Less than
>	Greater than
<=	Less than or equal
>=	Greater than or equal

Logical Precedence

AND has higher precedence than OR. Therefore, an expression such as A or B and C is interpreted as A or (B and C).

Return Fields and _score

```
"<field>", "<field>", ..., _score
```

Example:

```
"Name", "text", _score
```

_score is the raw metric result produced by the vector search. For similarity metrics, higher values indicate better matches. For distance metrics, lower values indicate better matches. The ordering of find automatically follows the metric's natural ordering.

Column Properties and Data Types

Column Properties

Property	Meaning
primary	Marks the field as the primary key.
auto_increment	Automatically increments the value.
nullable	Allows the field to contain a null value.

Scalar Data Types

Type	Meaning
text	Variable-length text.
int32	32-bit signed integer.
int64	64-bit signed integer.
uint32	32-bit unsigned integer.
uint64	64-bit unsigned integer.
fp16	16-bit floating point.
fp32	32-bit floating point.
fp64	64-bit floating point.
fp128	128-bit floating point.
datetime	ISO 8601 timestamp, e.g. 2026-09-16T10:00:00Z.
bool	Boolean value: 0 or 1.
bytes	Raw binary data, such as image data.

Vector Data Type

```
vector<type, size>
```

The vector element types defined by the current specification are: binary, int4, int8, int16, fp16, fp32, fp64. The size is specified by the user.

Change: fp128 remains available as a scalar data type, but it is not listed as a vector element type. This distinction is now explicit rather than leaving the reader to infer it.

Vector Metrics

Metric	Description	Typical Use
cosine	Angle between vectors.	Text embeddings, semantic search.
dot_product	Alignment plus magnitude.	Many embedding models, recommendation.
euclidean	Straight-line distance.	Clustering, spatial / vector data.

A vector column has one native metric. The metric is declared in the vector column definition and is used by `find ... within ...` when no separate metric is specified.

```
"doc_embedding" vector<fp32, 768> metric cosine
```

Reserved Names

Fields prefixed with `_` are engine-reserved and cannot be used as user column names.

Currently defined reserved name: `_score`, the computed distance / similarity value returned by a `find` operation.

Core Semantic Rules

Rule	Specification
Collection terminology	A collection is the DQL term for a table-like logical data container.
Vector metric	Every vector column has one native metric, declared in its vector type definition.
Index cardinality	A vector column has at most one active vector index.
Multi-column CREATE INDEX	Multiple vector columns in one statement create independent per-column indexes.
ANN search properties	Search parameters must match the target index type; unsupported parameters cause the query to fail.
Filter precedence	AND binds more tightly than OR.
<code>_score</code>	The raw metric result; ordering follows the metric's natural ordering.
Drop collection	Removes the collection definition and leaves its former data unclaimed for later reclamation.
Top clause	The current grammar requires an explicit top clause; no implicit result-count default is introduced.